

I have strong experience in AWS architecture, serverless design, infrastructure automation. I design secure and scalable cloud architectures using AWS services.

In my last project, I worked as an AWS Cloud Architect for a UK-based general insurance organization that provided motor, travel, home, and commercial insurance. The client operated customer portals through which users generated quotations, purchased and renewed policies, paid premiums, registered claims, uploaded supporting documents, and tracked claim status.

I designed a phased AWS modernization architecture using Amazon ECS with AWS Fargate. The application was organized into independently deployable services covering customer management, policy quotation, policy administration, claim registration, documents, payments, and notifications.

I used Route 53, CloudFront, AWS WAF, and an Application Load Balancer for secure external access. ECS tasks ran in private subnets across multiple Availability Zones and scaled automatically based on workload demand. Aurora PostgreSQL supported policy and claims transactions, while S3 stored policy and claim documents. Secrets Manager, KMS, security groups, and ECS task roles provided secure access and credential management.

In my AWS infrastructure project, I created CI/CD automation for both infrastructure provisioning and application deployment. Infrastructure was managed using Terraform, and application deployment was automated through a CI/CD pipeline. Whenever code was committed to Git, the pipeline performed validation, build, security checks, Terraform plan/apply for infrastructure changes, and then deployed the application to AWS services like Lambda or ECS. I used separate environments like dev, stage, and prod with environment-specific variables. This helped us reduce manual console changes, maintain version control, improve rollback capability, and ensure consistent infrastructure across environments.

For automation, I created Docker-based deployment flow. Docker file mein application runtime, dependencies, source code copy, package installation, port exposure, and server start command define kiya. Then CI/CD pipeline automatically built the Docker image, tagged it, pushed it to Amazon ECR, and deployed the updated image to ECS Fargate. Isse deployment repeatable, consistent aur environment-independent ho gaya.

Part 1: Application architecture

“I implemented an insurance platform with a React-based frontend hosted on Amazon S3 and delivered through Amazon CloudFront.

The frontend communicated with independently deployed Claim and Policy microservices through an Application Load Balancer using path-based routing.

I deployed both microservices as separate Amazon ECS Fargate services. Each service had its own ECR repository, Docker image, ECS task definition, target group, health checks and CloudWatch logs. This allowed both microservices to be built, deployed and scaled independently.”

Part 2: DevSecOps workflow

I implemented a GitHub Actions-based DevSecOps pipeline. Developers worked on feature branches, added application code and unit tests, and raised pull requests to a protected main branch.

The pull-request pipeline performed linting, unit testing, code coverage validation, SonarCloud quality analysis, secret scanning, Python static security testing, dependency vulnerability scanning and Terraform security validation.

If any mandatory quality or security check failed, the pull request could not proceed to deployment. After approval and merge, the pipeline built the Docker image, generated an SBOM and scanned the image using Trivy.

The approved image was pushed to Amazon ECR and deployed to the DEV ECS Fargate service. ECS started the new tasks, and the Application Load Balancer health checks verified whether the application was healthy before the deployment was considered successful.

CloudWatch logs and metrics were used for deployment monitoring and troubleshooting. The overall promotion model was designed to promote the same approved Docker image from DEV to QA and then to PROD, with environment-specific approvals and IAM roles.

Part 3: Security tools

For security controls, I used Gitleaks to identify exposed secrets, Bandit for Python static application security testing, pip-audit for vulnerable Python dependencies, Checkov for Terraform security misconfigurations and Trivy for Docker image vulnerability scanning.

I also generated a CycloneDX SBOM to maintain visibility of the application's software components. After deployment to the test environment, I used OWASP ZAP to perform dynamic security testing against the running APIs.

Part 4: Secure AWS authentication with OIDC

For AWS authentication, I avoided storing permanent AWS access keys in GitHub. I configured GitHub as an OIDC identity provider in AWS and created an environment-specific IAM role.

The role's trust policy restricted role assumption to the approved GitHub repository and environment.

During deployment, GitHub Actions requested a short-lived OIDC token and presented it to AWS STS.

AWS STS validated the token claims against the IAM trust policy and issued temporary role credentials. The workflow used those temporary credentials to push the Docker image to Amazon ECR, register or update the ECS task definition and update the ECS service.

The permissions policy followed least privilege and allowed access only to the required DEV resources. For multi-environment deployment, I designed separate IAM roles and protected GitHub environments for DEV, QA and PROD.

Interview Answers:

I implemented a GitHub Actions-based DevSecOps pipeline for an insurance platform. The platform had a React frontend hosted on Amazon S3 and separate Claim and Policy microservices deployed on Amazon ECS Fargate. The frontend accessed both microservices through an Application Load Balancer using path-based routing.

Developers worked on feature branches, added code and unit tests, and raised pull requests to a protected main branch. On every pull request, the CI pipeline performed Ruff linting, Pytest unit testing, coverage validation, SonarCloud quality analysis, Gitleaks secret scanning, Bandit SAST, pip-audit dependency scanning and Checkov Terraform security validation.

If any mandatory check failed, the pipeline stopped and the pull request was not ready for deployment.

After approval and merge, the CD pipeline built the Docker image, generated a CycloneDX SBOM and scanned the image using Trivy. The approved image was then pushed to Amazon ECR and deployed to the DEV ECS Fargate service.

For AWS authentication, I did not store permanent access keys in GitHub. I configured GitHub OIDC with an environment-specific IAM role. GitHub Actions presented its short-lived OIDC token to AWS STS, which validated the IAM trust policy and issued temporary credentials. The pipeline used those credentials to push the image to ECR and update only the required DEV ECS resources.

ECS started the new tasks, and Application Load Balancer health checks validated their health. I used CloudWatch logs and metrics for monitoring and troubleshooting. The promotion model was designed to deploy the same approved image from DEV to QA and then to PROD using independent IAM roles, protected environments and manual production approval.

My responsibility was to automate the CI/CD workflow, security gates, IAM access, image promotion, infrastructure validation, deployment monitoring and rollback controls. Developers were responsible for application code, unit tests and fixing application-level security or quality findings.

GitHub proves identity using OIDC

AWS verifies identity using OIDC Provider

Trust Policy allows only our repository and DEV

AWS STS gives temporary credentials

Permissions Policy decides allowed AWS actions

GitHub Actions se AWS deployment ka easy flow:

```
Developer pushes code
↓
GitHub Actions pipeline starts
↓
Code quality and security checks run
↓
Docker image is created and scanned
↓
GitHub requests an OIDC token
↓
AWS STS validates the token and IAM trust policy
↓
AWS provides temporary credentials
↓
Docker image is pushed to ECR
↓
ECS service is updated with the new image
↓
Health checks verify the deployment
```

Automatic rollback interview answer

For deployment rollback, I configured the Amazon ECS deployment circuit breaker with automatic rollback enabled.

Whenever a new Docker image was deployed, ECS created new tasks using a new task-definition revision. During the rolling deployment, ECS checked whether the new tasks reached the running state and passed the container and Application Load Balancer health checks.

If the new tasks failed to start, continuously crashed or failed their health checks, the deployment circuit breaker marked the deployment as failed. ECS then automatically rolled the service back to the last successfully completed task-definition revision.

I also used CloudWatch logs and metrics to investigate the root cause, such as an application startup error, incorrect environment variable, insufficient CPU or memory, port mismatch or failed downstream-service connectivity.

This ensured that unhealthy application versions did not replace the last stable version.

CI/CD pipeline ka complete flow:

Developer commits code | Pull Request created | Pipeline starts | Code quality/security checks | Docker image build | Container scan | Login to ECR using IAM role/OIDC | Push image to ECR with version tag | Terraform plan/apply or deploy job | New ECS task definition revision | Update ECS service | Rolling deployment | ALB health check validates new task | Old task drains | Route 53 domain already points to ALB | Production traffic served

CI/CD mein YAML file pipeline ko instructions deti hai, aur CI/CD tool un instructions ko execute karke application AWS par deploy karta hai. YAML file khud deployment nahi karti. GitHub Actions, AWS CodePipeline/CodeBuild, jaisa tool YAML ko read karke commands run karta hai. YAML file deployment instructions define karti hai. CI/CD tool YAML ko read karke AWS credentials aur IAM permissions ke through application ko AWS services par deploy karta hai.

Real project mein Terraform modules aise hote hain:

```
Infra/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
└── modules/
    ├── vpc/
    └── ecr/
```

- └─ alb/
- └─ ecs/
- └─ iam/
- └─ security-groups/
- └─ route53/

Terraform pipeline mein ye stages hoti hain:

terraform fmt, terraform validate, terraform plan, manual approval, terraform apply

Monitoring and logs: I use CloudWatch Logs, CloudWatch Metrics, Container Insights, ALB access logs, X-Ray optional, CloudTrail, ECS CPU/memory, Task restart count, Desired vs running tasks, ALB target health, Target response time, Deployment failures, Application logs

Infra Design Explanation: 1. Business requirement 2. High-level architecture 3. Networking design 4. Compute design 5. Database design 6. Security design 7. Monitoring and logging 8. CI/CD and automation 9. Backup and DR 10. Cost optimization

Security:

For security, I used IAM role/OIDC instead of storing long-term AWS keys.

How did you implement IAM security?

I applied least-privilege access at the workload and pipeline levels. Each ECS application service received a separate task role. For example:

The document service could access only the required S3 document paths.

The notification service could publish only to approved notification resources.

The claims service could access only the secrets and resources required for claims processing.

I did not share one broad IAM role across all application services. AWS recommends separate task roles because they provide granular permissions and align with least-privilege access

What is the difference between an IAM policy and an IAM role?

An IAM policy is a JSON permissions document that defines which actions are allowed or denied on which resources. An IAM role is an AWS identity with a trust policy and attached permission policies. A trusted entity, such as an ECS task, EC2 instance, Lambda function, or CI/CD pipeline, assumes the role and receives temporary credentials. I attached tightly scoped policies to roles instead of storing permanent access keys inside applications.

How did you handle secrets during deployment?

Secrets were not stored in Git, Terraform variable files, Docker images, or pipeline logs. Terraform created or referenced the secret resources without exposing secret values in source control. The ECS task definition referenced Secrets Manager, and authorized workloads received the values at runtime. I also controlled who could update, read, and rotate secrets through separate IAM permissions.

How would you manage and secure secrets for a CI/CD pipeline in AWS?

To securely manage secrets in an AWS CI/CD pipeline, you can use AWS Secrets Manager or AWS Systems Manager Parameter Store to store sensitive information such as API keys, database passwords, and certificates. Both services integrate with AWS services like CodePipeline and CodeBuild, allowing secure access to secrets without hardcoding them in your codebase.

By controlling access permissions with IAM, you can ensure that only authorized entities can access sensitive data, enhancing security within the CI/CD process.

DDoS stands for Distributed Denial of Service. In this attack, multiple compromised devices generate a huge amount of traffic against an application, making it slow or unavailable for legitimate users. In AWS, I use services such as Shield, WAF, CloudFront, and Route 53 to improve protection and availability.

EC2 application experiences traffic spikes only during business hours. How would you optimize cost and performance?

Main application ko ek single EC2 instance par host nahi karunga. Main:

1. **Application Load Balancer** configure karunga
 2. EC2 instances ko **Auto Scaling Group** ke andar deploy karunga
 3. Minimum, desired aur maximum capacity define karunga
 4. Business hours ke liye **scheduled scaling** configure karunga
 5. Unexpected traffic spike ke liye **target tracking scaling** bhi rakhunga
 6. Non-critical stateless workloads ke liye suitable ho toh Spot Instances use karunga
-

Optimization:

How did you optimize costs?

I optimized cost at architecture, workload, and operational levels:

- Right-sized Fargate CPU and memory using observed utilization
- Used ECS auto scaling to reduce idle capacity
- Maintained only the required minimum task count
- Used scheduled scaling for predictable business peaks
- Evaluated Fargate Spot only for interruption-tolerant workloads
- Right-sized legacy EC2 instances
- Used Savings Plans for stable baseline compute where appropriate
- Applied S3 lifecycle policies to older documents
- Defined CloudWatch log-retention periods
- Applied ECR lifecycle policies to old images
- Right-sized Aurora capacity
- Shut down or scaled down non-production resources outside required hours
- Used AWS Cost Explorer, budgets, tagging, and cost-allocation controls

How do you optimize AWS costs for a high-traffic web application?

To optimize AWS costs for a high-traffic application, you can start by using AWS Cost Explorer and AWS Budgets to monitor and manage spending. Then, consider these strategies:

- **Use Reserved and Spot Instances** for predictable and flexible workloads, respectively.
- **Auto-scaling** helps adjust resource allocation based on demand, reducing costs during low-traffic periods.
- **Optimize storage** with Amazon S3 lifecycle policies and S3 Intelligent-Tiering to move infrequently accessed data to cost-effective storage classes.
- **Implement caching** with Amazon CloudFront and Amazon ElastiCache to reduce repeated requests to backend resources, saving bandwidth and compute costs.

How did you track project cost?

I implemented a consistent tagging strategy across services, including application, environment, business owner, cost center, and managed-by information. I used cost reports, budgets, and service-level analysis to compare production, non-production, compute, database, storage, monitoring, and data-transfer costs.

Explain the VPC design.

I designed the VPC across multiple Availability Zones with separate network layers:

Public subnets for the internet-facing Application Load Balancer

Private application subnets for ECS Fargate tasks

Private or isolated database subnets for Aurora

Only the load-balancing layer accepted inbound internet traffic. ECS tasks, EC2 application servers, and Aurora were not directly exposed to the internet.

VPC

| — Availability Zone A

| | — Public Subnet: ALB

| | — Private App Subnet: Fargate tasks

| — Private DB Subnet: Aurora
|
| — Availability Zone B
| — Public Subnet: ALB
| — Private App Subnet: Fargate tasks
| — Private DB Subnet: Aurora

What is Terraform State?

Terraform state stores the mapping between infrastructure resources and Terraform configuration. I store Terraform state in a secure and versioned S3 remote backend so that developers and CI/CD pipelines use the same centralized state. I use S3 native locking with `use_lockfile = true`.

When one Terraform operation is modifying infrastructure, it acquires the lock. Any second developer or pipeline attempting a conflicting operation cannot acquire the lock and Terraform stops that operation, preventing concurrent state writes. I also maintain separate state keys for DEV, QA, and PROD and allow production apply only through an approved CI/CD pipeline.

- Terraform ne kaunse AWS resources manage kiye?
- Unke AWS resource IDs kya hain?
- Terraform code aur actual AWS infrastructure ka mapping kya hai?
- Agli plan/apply mein kya add, change ya delete karna hai?

Follow-up: “What was your personal contribution?”

I gathered the functional and non-functional requirements and translated them into the AWS target architecture. I personally defined the VPC segmentation, external traffic flow, ECS Fargate architecture, service boundaries, EC2 exception criteria, IAM model, security groups, secret-management approach, encryption standards, autoscaling strategy, high-availability controls, monitoring requirements, and disaster-recovery approach.

I also reviewed Terraform modules, ECS task definitions, IAM policies, deployment pipelines, and production-readiness checklists. I provided technical direction across application, DevOps, security, networking, and operations teams.

What is Amazon ECS?

Amazon ECS is AWS’s managed container-orchestration service. I used it to deploy, manage, monitor, and scale containerized insurance services such as quotation, policy, claims, payment, document, and notification services. ECS maintained the required number of application tasks, replaced unhealthy tasks, integrated with the Application Load Balancer, and supported service-level auto scaling.

What is the difference between ECS and Fargate?

ECS is the container-orchestration service, while Fargate is a compute option on which ECS tasks can run. ECS manages services, deployments, task definitions, desired task counts, health, and integration with the load balancer. Fargate provides serverless compute capacity for those containers, so I did not need to provision or manage the underlying EC2 container-host instances.

Why did you select ECS Fargate instead of EKS?

I evaluated the orchestration requirement before selecting the service. The client required independent deployments, container health management, automatic scaling, load-balancer integration, and high availability. The client did not require Kubernetes operators, Kubernetes-specific tooling, custom schedulers, or portability across Kubernetes environments. The operations team also wanted to minimize platform-management overhead. Therefore, I selected ECS with Fargate because it met the business and technical requirements with lower operational complexity than EKS.

What is an ECS task definition?

I treated the ECS task definition as the deployment blueprint for a containerized service. It defined: Docker image location, CPU and memory, Container port, Environment configuration, Secrets references, CloudWatch logging, Task execution role, Application task role, Health-check related configuration.

What is the difference between an ECS task and ECS service?

A task is a running instance of a task definition. A service maintains the required number of tasks and replaces failed or unhealthy tasks. For example, if the claims service required a minimum of two tasks and one task failed, the ECS service launched a replacement task to restore the desired count.

How did the application receive internet traffic?

Route 53 resolved the application domain. CloudFront provided the controlled edge entry point, and AWS WAF inspected incoming web requests. Traffic was forwarded to the Application Load Balancer, which terminated HTTPS using an ACM certificate and sent requests to healthy ECS targets. The load balancer used host-based or path-based rules to route requests to the appropriate insurance service.
Customer->Route 53->CloudFront->AWS WAF->Application Load Balancer->ECS Fargate

How did you implement autoscaling?

I implemented ECS Service Auto Scaling for individual business services.

I configured minimum, desired, and maximum task counts and used target-tracking policies based on CPU, memory, and Application Load Balancer request count, depending on the workload behavior. For predictable renewal campaigns, scheduled scaling could increase the minimum capacity before the expected traffic arrived. ECS publishes average CPU and memory metrics through CloudWatch.

What is the difference between CloudWatch and CloudTrail?

CloudWatch is primarily for operational visibility through metrics, logs, alarms, and dashboards.

CloudTrail records AWS API activity and supports security auditing and investigation.

For example, an ECS task CPU alarm belonged in CloudWatch. To identify who modified a security group or IAM policy, I used CloudTrail.

What is RTO?

RTO, or Recovery Time Objective, is the maximum acceptable time between a service interruption and restoration of the service. If the claims portal has an RTO of two hours, the recovery process must restore that service within two hours of the qualifying disaster. AWS defines RTO as the maximum acceptable delay between service interruption and restoration.

What is RPO?

RPO, or Recovery Point Objective, defines the maximum acceptable data-loss window.

If the policy database has an RPO of 15 minutes, the recovery design must ensure that the organization does not lose more than approximately 15 minutes of eligible data following a disaster.

AWS defines RPO as the maximum acceptable time since the last recoverable data point.

What did you back up?

I included:

- Aurora automated backups and point-in-time recovery
- Database snapshots according to retention policy
- S3 versioning and lifecycle controls for eligible documents
- EBS snapshots for required legacy EC2 data volumes
- Terraform source code and environment configuration
- Application source and deployment definitions
- Versioned Docker images in ECR according to retention requirements

What is blue-green deployment?

Blue represented the currently running production version, and green represented the new application version.

The green tasks were deployed separately and validated through health checks and test traffic. After validation, production traffic shifted from blue to green.

If the new version failed the defined checks, traffic could be returned to blue.

Amazon ECS blue-green deployment runs the current and new service revisions separately, supports validation before shifting production traffic, and provides a quicker rollback path.

Why use blue-green instead of rolling deployment?

I selected the deployment method according to workload criticality and cost.

Rolling deployment uses fewer additional resources and is suitable for many services, but old and new versions can run together during the rollout.

Blue-green provides stronger isolation, controlled traffic shifting, and easier rollback, but temporarily runs both environments and therefore costs more.

I preferred blue-green for critical customer-facing services such as quotation, policy purchase, and claims registration. Less critical internal services could use controlled rolling deployment.

How did rollback work?

The previous ECR image and ECS task-definition revision remained available.

For blue-green deployment, traffic returned to the previous target group. For a rolling deployment, the ECS service was updated to the previous validated task-definition revision. Database changes required backward-compatible migration planning because rolling back application code does not automatically reverse database-schema changes.

In Blue-Green deployment, I maintain two complete application versions. The existing Blue version serves production traffic while the Green version is deployed and validated separately. After validation, I switch the production Service from Blue to Green. This provides a simple and fast rollback by redirecting the Service back to Blue.

In Canary deployment, I gradually expose the new version to production traffic, for example 5 percent, 20 percent, 50 percent and finally 100 percent. I monitor technical and business metrics at every stage. Canary reduces the blast radius because only a small percentage of users initially access the new version, whereas Blue-Green generally performs a complete traffic cutover after validation.

My CI/CD pipeline built the Docker image, performed testing and scanning, and pushed the versioned image to Amazon ECR. The pipeline updated the Green deployment in EKS using Helm. After readiness checks and smoke testing, the deployment required an approval before switching the production Service from Blue to Green. After promotion, I monitored operational metrics and retained Blue for immediate rollback.

For an RPO of zero, I use synchronous data replication so that a transaction is acknowledged only after the required durable copies are updated. On AWS, RDS Multi-AZ can provide zero-data-loss failover for committed transactions. However, I do not treat synchronous replication as a backup. I also use database transactions, idempotency controls, transactional outbox patterns, point-in-time recovery and tested backups to protect the complete business workflow.

Data-protection strategy:

1. RDS Multi-AZ -> Instance/AZ failure se protection
2. Automated backups -> Historical recovery
3. Point-in-time recovery -> Particular time tak database restore
4. Cross-Region backups -> Regional disaster protection
5. AWS Backup -> Centralized backup policies
6. Restore testing -> Verify backup actually usable hai

AWS Ki Four Main DR Strategies: Backup and Restore, Pilot Light, Warm Standby, Multi-Site Active-Active

I select the disaster recovery strategy based on the business-defined RTO and RPO. AWS provides four primary DR approaches: Backup and Restore, Pilot Light, Warm Standby, and Multi-Site Active-Active. For a business-critical insurance application, I use a Warm Standby architecture across two AWS Regions. The primary Region runs at full capacity, while the DR Region contains the complete application stack at reduced capacity. I continuously replicate the database, S3 attachments, container images, secrets, and required configuration to the DR Region.

If the primary Region fails, health monitoring detects the failure, the secondary database is promoted, compute capacity is scaled, and Route 53 or Global Accelerator redirects traffic to the healthy Region. I maintain Infrastructure as Code, automated backups, CloudWatch alarms, SNS notifications, a documented runbook, and regular DR drills.

I also maintain backups because replication alone cannot protect against accidental deletion or data corruption. After the primary Region becomes stable, I synchronize the latest data and perform a controlled failback.

How do you use AWS Systems Manager in a production environment?

AWS Systems Manager helps automate and manage your infrastructure at scale. In a production environment, it's commonly used for patch management, remote command execution, inventory collection, and securely storing configuration parameters and secrets. It integrates with EC2, RDS, and other AWS services, enabling centralized visibility and operational control.

Maan lo production mein aapke paas 100 EC2 servers hain. Har server par manually SSH karke commands chalana, patches install karna aur configuration check karna difficult aur insecure hoga.

Systems Manager se aap:

- SSH port khole bina server mein login kar sakte ho
- Multiple EC2 instances par ek saath commands chala sakte ho
- OS security patches install kara sakte ho
- Application configuration aur parameters store kar sakte ho
- Operational tasks automate kar sakte ho
- Installed software aur instance information dekh sakte ho

Systems Manager se server manage karne ke liye normally teen important cheezein chahiye:

1. SSM Agent
2. IAM Role
3. Network connectivity

What are the strategies to create a highly available and fault-tolerant AWS architecture for critical web applications?

I design highly available and fault-tolerant AWS applications by first defining the availability target, critical dependencies, RTO, and RPO. I then eliminate single points of failure by deploying the workload across at least two Availability Zones.

I use Route 53, CloudFront, and an Application Load Balancer to route traffic to healthy application instances or containers. At the compute layer, I use EC2 Auto Scaling, ECS Fargate, or EKS with health checks, horizontal scaling, and automatic replacement of unhealthy resources.

I keep application services stateless and store state in resilient managed services. For relational databases, I use RDS Multi-AZ or Aurora with automatic failover. I also configure automated backups, point-in-time recovery, and cross-Region backup copies where required.

To prevent cascading failures, I decouple services using SQS, SNS, or EventBridge and implement timeouts, retries with exponential backoff, circuit breakers, idempotency, and dead-letter queues.

I monitor application and infrastructure health using CloudWatch metrics, logs, dashboards, alarms, tracing, ALB health checks, and synthetic monitoring. For deployments, I use rolling, Blue/Green, or canary releases with automated validation and rollback.

Multi-AZ is my default design for Regional high availability. If the business cannot tolerate a complete Regional outage, I select an appropriate multi-Region disaster-recovery strategy based on RTO, RPO, cost, and complexity. I also regularly test failover, backup restoration, and recovery runbooks because an untested recovery strategy cannot be considered reliable.

Amazon RDS is ideal for applications that require a traditional relational database with standard SQL support, transactions, and complex queries.

Amazon DynamoDB suits applications needing a highly scalable, NoSQL database with fast, predictable performance at any scale. It's great for flexible data models and rapid development.

Amazon Redshift is a fully managed cloud data warehouse used for large-scale analytics and reporting. Its major benefits include columnar storage, massively parallel query processing, scalability, integration with BI tools and the ability to query data stored in Amazon S3 through Redshift Spectrum. In an insurance architecture, I use Aurora or DynamoDB for real-time claim and policy transactions, while data is moved to Redshift through an ETL pipeline for historical analysis, fraud reporting, trend analysis and management dashboards.

Redshift ka primary benefit ye hai ki hum bahut large historical data par fast SQL analytics aur business reporting kar sakte hain, bina transactional application database par reporting load dale.

RDS application transactions ke liye hota hai, jabki Redshift large-scale reporting aur analytics ke liye hota hai.

Redshift = bahut saare historical data ki reporting aur analytics

Redis = application ko bahut fast banane ke liye temporary data memory mein rakhna

What considerations would you take into account when migrating an existing on-premises application to AWS? Use an example of choice.

AWS recommends thinking about migration through the **"7 Rs" framework**: rehost (lift and shift), replatform, repurchase, refactor, retire, retain, and relocate. The right choice depends on the application's complexity and how much you're willing to change during the move. Take a CRM migration as an example. Key considerations:

Strategy. A custom-built CRM often suits *replatform* — moving to managed services like RDS and EC2 without rewriting code. A heavily customized commercial CRM might call for *repurchase* (switching to a SaaS equivalent) instead.

- **Data migration.** Use AWS DMS to move the database with minimal downtime, and Direct Connect to keep the network link stable during cutover.
- **Dependencies.** Catalog what the CRM connects to (email, billing, support) and migrate non-critical modules first to validate the approach.
- **Security and compliance.** Replicate access controls with IAM, encrypt data with KMS, and confirm regulatory requirements (GDPR, HIPAA) carry over.
- **Cost.** Model post-migration cost with AWS Pricing Calculator and plan for the shift from capex to opex.

What is ElastiCache?

Amazon ElastiCache ek fully managed, in-memory caching service hai, jo frequently accessed data ko RAM mein temporarily store karta hai, taaki application ko baar-baar RDS, Aurora ya kisi main database ko query na karna pade. AWS currently ElastiCache mein **Valkey, Redis OSS aur Memcached** engines support karta hai. Redis engine hai, ElastiCache AWS ki managed service hai jo Redis-compatible cache ko operate aur manage karti hai.

What are the key pillars of the AWS Well-Architected Framework?

- **Operational excellence:** Focuses on supporting development and operations through monitoring, incident response, and automation.
- **Security:** Covers protecting data, systems, and assets through identity management, encryption, and incident response.
- **Reliability:** Involves building systems that can recover from failures, scaling resources dynamically, and handling network issues.
- **Performance efficiency:** Encourages the use of scalable resources and optimized workloads.
- **Cost optimization:** Focuses on managing costs by selecting the right resources and using pricing models such as Reserved Instances.

- **Sustainability:** AWS ka newer pillar: Example: Right sizing, Serverless, Auto Scaling

How would you ensure high availability for a database in AWS?

For high availability, I would use **Amazon RDS Multi-AZ** deployments, which automatically replicate the database to a standby instance in a different availability zone. This ensures automatic failover in case of a failure. For NoSQL databases, I would use **Amazon DynamoDB** with **Global Tables** to replicate data across multiple regions. Additionally, I would set up **read replicas** for scaling read operations.

I use RDS Read Replicas when the application has a read-heavy workload and read queries are increasing CPU or I/O load on the primary database. I keep insert, update and delete operations on the primary writer endpoint, while I route suitable queries such as dashboards, searches and reports to the replica endpoint. Because RDS Read Replicas normally use asynchronous replication, I consider replication lag and send consistency-sensitive reads, such as an immediate confirmation after a write, back to the primary. For production availability I use Multi-AZ, while for read scalability I use Read Replicas. Both can be used together.

How do you manage and secure sensitive data such as passwords in AWS?

I would use **AWS Secrets Manager** or **AWS Systems Manager Parameter Store** to securely store sensitive data like API keys, database passwords, and other credentials. These services provide encryption at rest and can be easily accessed by applications using IAM roles. Additionally, I would implement **IAM policies** to restrict access to these secrets based on least-privilege principles.

How would you use Amazon CloudWatch to monitor your application's health?

I would set up **CloudWatch metrics** for EC2 instances, RDS databases, Lambda functions, and other AWS resources to monitor their performance. I would configure **CloudWatch Alarms** to alert when a metric breaches a threshold, such as high CPU utilization or low disk space. I would also set up **CloudWatch Logs** to collect and analyze log data from EC2 instances and applications.

How do you ensure the security of EC2 instances in AWS?

I would follow security best practices such as:

- Using **Security Groups** and **Network ACLs** to control inbound and outbound traffic to the EC2 instances.
- Ensuring that only required ports are open and using **SSH keys** for secure access.
- Installing and configuring **AWS Systems Manager** for patch management and security updates.
- Enabling **CloudTrail** to monitor API activity and implement **IAM roles** with the least privilege.

How would you secure a REST API hosted on AWS API Gateway?

I would secure the API using **AWS IAM roles and policies** to restrict access to authorized users. Additionally, I would implement **AWS WAF (Web Application Firewall)** to protect against common web exploits and configure **API keys** or **Cognito User Pools** for user authentication. I could also integrate **Lambda authorizers** for custom authentication and authorization logic.

How do you configure VPC peering between two VPCs in different AWS accounts?

To configure **VPC peering**, I would create a peering connection request from one VPC and accept the request in the other VPC. I would update the **route tables** in both VPCs to route traffic to the peered VPC. Additionally, I would modify **Security Groups** and **Network ACLs** to allow communication between instances in the two VPCs.

How do you set up cross-account access in AWS using IAM roles?

To set up cross-account access, I would create an **IAM role** in the target account with the necessary permissions. In the source account, I would create an **IAM policy** allowing users or services to assume the role in the target account using the `sts:AssumeRole` action. The user or service in the source account would use **AWS CLI** or SDKs to assume the role.

How do you scale an Amazon RDS database to handle increased traffic?

I would use **Amazon RDS Read Replicas** to offload read traffic from the primary database. Additionally, I would use **Amazon RDS Multi-AZ** deployments for automatic failover and high availability. If more storage is needed, I would resize the RDS instance to a larger type or scale horizontally by adding more read replicas.

You want to connect your on-premises network to AWS securely. What do you use?

Use **AWS Site-to-Site VPN** or **AWS Direct Connect**.

VPN: Quick setup, encrypted over the internet

Direct Connect: Dedicated line, low latency, higher cost

Your EC2 in a private subnet needs internet access. What's the solution?

Use a NAT Gateway in a public subnet and update the route table of the private subnet to send 0.0.0.0/0 through the NAT.

How can two VPCs communicate without public IPs?

Use **VPC Peering**.

Setup peering

Add routes to both route tables

Ensure no CIDR block overlap

You created a VPC, but instances can't access the internet. Why?

Possible reasons:

No Internet Gateway attached

Missing route to 0.0.0.0/0

Security Group or NACL blocking traffic

You need to isolate a subnet from others in the same VPC. What do you do?

Use custom route tables, NACLs, and security groups to control access. Don't allow internal routes if isolation is required.

Can two subnets have overlapping CIDRs?

No. Subnets within the same VPC must have unique, non-overlapping CIDRs.

How do you connect VPCs across different AWS regions?

Use **Inter-Region VPC Peering**.

- Lower latency than VPN
- No internet traversal
- Still uses private IPs

How do you isolate workloads in the same VPC?

- Use different subnets
- Set strict NACLs
- Apply role-specific security groups
- Separate route tables if needed

How to allow SSH access only from your office IP?

Edit Security Group:

Allow TCP port 22 from your static office IP address (e.g., 203.0.113.10/32).

Can a VPC have multiple Internet Gateways?

No. A VPC can only have one Internet Gateway attached at any time.

What does a route table do?

It defines how traffic flows:

- Between subnets
- To the internet (via IGW)

- To other VPCs (via peering or VPN)

Your NAT Gateway stopped working. What could be wrong?

Check:

- NAT is in a public subnet with IGW
- Elastic IP is not released
- Route table correctly points 0.0.0.0/0 to NAT

What's the default VPC limit per region?

5 VPCs per region per account (soft limit – can be increased via a service quota request).

How to restrict EC2 access to S3 over VPC only?

- Create a VPC Endpoint for S3
- Add a bucket policy allowing access only from that VPC endpoint

What is the shared responsibility model?

The shared responsibility model defines security roles between AWS and the customer. AWS secures the cloud infrastructure, hardware, and global network. Customers are responsible for security in the cloud, including data, access control, encryption, and application configurations.”

What are the different types of cloud computing models (IaaS, PaaS, SaaS)?

IaaS (Infrastructure as a Service): AWS offers virtual servers and networking resources. You can use it to build and run your own applications. You don't have to worry about buying or managing physical computers; AWS takes care of it for you. IaaS offers you the infrastructure (servers, networks).

PaaS (platform as a service): AWS PaaS services manage the infrastructure, runtime, and scaling. Your focus is only on writing and deploying your app. PaaS gives you a platform to develop apps.

SaaS (Software as a Service), or SaaS, eliminates the need for installation. Just log in and start using the software directly. A SaaS application gives you ready-to-use software.

What is AWS KMS (Key Management Service)?

AWS Key Management Service functions as a security service within AWS. It helps users generate and manage encryption keys, which protect data stored in Amazon S3, Amazon EBS, and Amazon RDS. Organizations should use this solution to protect sensitive information stored in Amazon cloud environments.

You need to deploy a high-traffic website on AWS. The website sees spikes in traffic during special promotions. Design an infrastructure that ensures scalability, availability, and cost-effectiveness.

- For starters, set up Amazon EC2 instances for web servers and position them behind an Auto Scaling group. This handles varying traffic loads.
- Next, implement Amazon RDS for the database to increase data durability and control.
- To reduce latency, use Amazon CloudFront for content delivery. It also improves website performance.
- I'd use Amazon ElastiCache for content caching to reduce database load and boost performance.
- Then, install Amazon Route 53 for DNS routing with latency-based routing policies for global users.
- And for automated scaling triggers, use Amazon CloudWatch Alarms and AWS Lambda.
- Moreover, monitor performance using Amazon CloudWatch and AWS Trusted Advisor

What security measures protect data at rest and in transit in AWS S3?

Data at rest is secured by using server-side encryption. For in-transit SSL/TLS protects the data. Other measures include bucket policies, access controls, and IAM roles.

What is Amazon CloudFront?

Amazon CloudFront operates as a cloud-based platform content delivery network (CDN). Users can quickly and securely deliver websites, images, videos, and APIs to their users through this service.

Content caching takes place at numerous global edge locations to make it work. CloudFront minimizes latency, boosts load times, and speeds up access for worldwide audiences. The service decreases your main server workload while maintaining seamless integration with AWS services. CloudFront content ko edge par cache karta hai, jabki Global Accelerator user traffic ko AWS ke global network se fastest healthy application endpoint tak pahunchata hai.

Your application is deployed globally, but due to compliance or security reasons, you need to block access from specific countries while ensuring minimal disruption. How would you achieve this in AWS efficiently?

- a) AWS WAF (Web Application Firewall) allows you to block requests from certain countries using Geo-Match Rules. This method is useful if you want to completely deny access from restricted regions.
- b) Instead of blocking, you can redirect users from restricted countries to another page using AWS Route 53 Geolocation Routing.

How does Amazon Route 53 ensure high availability and low latency?

Route 53 selects Region based on DNS level. Route 53 works at the DNS level. It returns the endpoint of the Region expected to provide the lowest latency, after which the user's application traffic travels over the internet. Amazon Route 53 provides high availability through its globally distributed DNS infrastructure, so DNS resolution does not depend on a single server or location. If one location or network path has an issue, another available DNS location can process the query.

For low latency, I can configure latency-based routing when my application is deployed in multiple AWS Regions. Route 53 checks which Region provides the lowest latency for the user and returns that Region's endpoint. I also use Route 53 health checks and failover routing so that if the primary application endpoint becomes unhealthy, traffic is directed to a healthy endpoint in another Region.

AWS Global Accelerator

Improve availability and performance for your applications

AWS Global Accelerator is a service that improves the availability and performance of your applications with local or global users. It provides static IP addresses that act as a fixed entry point to your application endpoints in a single or multiple AWS Regions, and uses the AWS global network to optimize the path from your users to your applications.

Global Accelerator actual network traffic ko AWS global network ke through optimal Region tak pahunchata hai.

Global Accelerator works at the network layer. It provides global static anycast IP addresses, accepts user traffic at the nearest AWS Edge Location, and carries that traffic over the AWS global network to an optimal healthy regional endpoint. It also provides faster endpoint failover because the client continues using the same accelerator IP address, while Global Accelerator changes the backend destination.

I use Global Accelerator when I need global static IP addresses, optimized TCP or UDP traffic, more consistent global performance, and faster regional failover. I can also use both together by pointing a Route 53 alias record to Global Accelerator.

Global Accelerator also provides the optimized road to reach there.

Control Tower, AWS Organizations, Landing Zone explanation?

AWS Organizations = Accounts ko group aur centrally manage karne ka foundation

Landing Zone= Company ka ready, secure multi-account AWS environment

Control Tower = Landing Zone ko automatically set up aur govern karne wali AWS service

Organizations manages accounts.

Landing Zone is the complete environment.

Control Tower builds and governs that environment.

AWS Organizations provides the foundation for centrally managing multiple AWS accounts using organizational units and service control policies. However, I still need to design and configure the complete multi-account security, access, logging and governance environment.

A landing zone is that complete, preconfigured multi-account AWS environment. It includes the account and OU structure, centralized access, logging, audit accounts, security baselines and governance controls. AWS Control Tower helps me establish and govern this landing zone using AWS Organizations and other AWS services. It provides standardized controls, centralized logging and audit accounts, IAM Identity Center integration, Account Factory for provisioning governed accounts, and governance drift detection. Control Tower does not replace AWS Organizations; it adds an automation and governance layer on top of it.

1. Landing Zone ka exact meaning kya hai?

Company ke liye pehle se design aur configured secure multi-account AWS environment, jahan naye applications aur workloads safely deploy kiye ja saken.

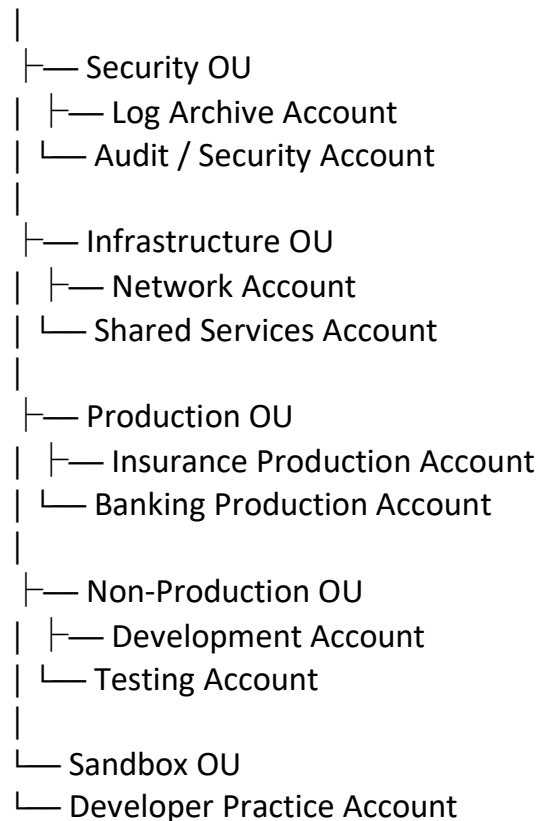
Isi tarah company AWS par applications deploy karne se pehle ek standard environment ready karti hai:

- AWS Organization
- Organizational Units
- Separate AWS accounts
- Centralized login
- Security rules
- CloudTrail logging
- AWS Config monitoring
- Audit account
- Log Archive account
- Approved AWS Regions
- Account provisioning process

In sabka combined ready environment **Landing Zone** kehlata hai.

Example landing zone

AWS Organization



2. AWS Organizations ka role

AWS Organizations basic structure provide karta hai:

- Management account
- Member accounts

- Organizational Units
- SCPs
- Centralized management
- Consolidated billing

Lekin AWS Organizations normally aapse ye architecture khud design aur configure karne ko kehta hai.

Aapko decide karna padega:

- Kaun se OUs honge?
- Security account kaise banega?
- Log Archive account kaise banega?
- CloudTrail logs kahan jayenge?
- AWS Config kaise enable hoga?
- Users multiple accounts mein login kaise karenge?
- Naya account standard configuration ke saath kaise create hoga?
- Configurations change hone par drift kaise detect hoga?

Yahin **AWS Control Tower** help karta hai.

AWS Control Tower ka role kya hai?

AWS Control Tower is a service that automatically sets up and manages a multi-account AWS environment. It helps create and govern a Landing Zone by following AWS best practices.

Control Tower works on top of AWS Organizations to create, manage, and monitor a secure and well-governed multi-account setup.

What is Redis?

Redis OSS is an in-memory data store commonly used for caching, session management, counters and rate limiting. On AWS, I can run a Redis OSS-compatible cache using Amazon ElastiCache, where AWS manages the underlying infrastructure, patching, monitoring, replication and scaling.

Logging, Monitoring and Threat Detection

CloudTrail → Who performed the AWS API action?

CloudWatch → Application or infrastructure mein kya ho raha hai?

Config → Resource configuration compliant hai?

GuardDuty → Suspicious activity ho rahi hai?

Security Hub → Saare findings ek jagah kaise dekhne?

Security Hub CSPM

Security Hub ek **central security management and prioritization layer** hai.

I used AWS Security Hub as the centralized security console. It consolidated findings from GuardDuty, Inspector, Macie and security posture controls, prioritized critical risks, and integrated with EventBridge for alerting and remediation workflows.

Security Hub findings ko centrally collect and prioritize karta hai and security standards/best-practice checks ke against posture visibility provide karta hai. GuardDuty findings Security Hub mein integrate kiye ja sakte hain.

GuardDuty finds a threat

Inspector finds a vulnerability

Macie finds sensitive data

Config identifies a configuration violation

↓

Security Hub

Central security console

Security Hub kya nahi karta?

- Ye har threat khud detect nahi karta

- EC2/ECR ko Inspector ki tarah scan nahi karta
- GuardDuty ki tarah raw activity analyze karke threat detection ka direct substitute nahi hai
- Detective ki tarah complete root-cause investigation tool nahi hai

Ye mostly **collect, correlate, prioritize aur orchestrate** karta hai.

IAM → Who can access what?

Cognito → Application user authentication

KMS → Encryption keys

Secrets Manager → Passwords and API secrets

WAF → Malicious web requests block

Shield → DDoS protection (Distributed Denial of Service)

Security Group → Resource-level network firewall

NACL → Subnet-level network control

CloudTrail → AWS API audit

CloudWatch → Logs, metrics and alarms

Config → Configuration compliance

GuardDuty → Suspicious activity detection

Inspector → Vulnerability scanning

Macie → Sensitive data discovery in S3

Security Hub → Central security findings

Detective → Security incident investigation

AWS Backup → Protected backups and recovery

Service	Simple role	Main question
GuardDuty	Threat detect karta hai	Kya account mein suspicious activity ho rahi hai?
Inspector	Vulnerability scan karta hai	Kya EC2, ECR image ya Lambda mein security weakness hai?
Macie	S3 mein sensitive data identify karta hai	Kya S3 mein PII ya confidential data pada hai?
AWS Config	Configuration compliance check karta hai	Kya resource security policy ke according configured hai?
Security Hub	Findings ko centralize aur prioritize karta hai	Sabse important security issues kya hain?
Detective	Security incident investigate karta hai	Attack kaise hua, kisne kiya aur kya impact hua?
CloudTrail	AWS API activity record karta hai	Kis identity ne kaunsa AWS action kiya?
CloudWatch	Logs, metrics aur alarms provide karta hai	Application aur infrastructure mein kya ho raha hai?

AWS trusted advisor

AWS Trusted Advisor ek automated AWS recommendation service hai jo aapke AWS account aur resources ko scan karke batata hai ki architecture mein cost, security, performance, availability ya service limits se related kya improvements karni chahiye.

Trusted Advisor AWS ka automated cloud consultant hai.

AWS Glue ek **serverless data integration aur ETL service** hai. Iska kaam multiple sources se data nikalna, clean/transform karna aur target system mein load karna hai. Infrastructure ya servers manually manage nahi karne padte

AWS Glue = serverless ETL/data integration service

Amazon EMR = very large-scale big-data processing platform

ETL ka meaning

E: Extract : Source se data nikalna:

RDS, DynamoDB, S3, Application database

T: Transform: Data ko usable banana:

Duplicate records remove karna, Incorrect values correct karna, Date format standardize karna, Multiple tables combine karna

Personal information mask karna

Unnecessary columns remove karna

L: Load: Processed data ko destination mein load karna:

S3, Redshift, Data Lake, Data Warehouse

1. SBOM kya hota hai?

SBOM = Software Bill of Materials

Docker image ya application ke andar kaun-kaun se software components, libraries aur versions present hain, uski complete inventory.

Software ki SBOM:

Insurance Claim Service

└─ Debian Linux packages

└─ Python 3.12

└─ FastAPI 0.141.1

└─ Uvicorn 0.35.0

└─ Pydantic

└─ OpenSSL

└─ Transitive dependencies

SBOM generate karne ka kya matlab hai?

Pipeline ek machine-readable file create karegi, commonly:

Sbom.cdx.json:

Common formats: CycloneDX, SPDX

Hum Trivy se roughly ye operation automate karenge:

Docker image build

↓

Trivy image inspect karta hai

↓

Image ke components identify karta hai

↓

SBOM JSON file generate hoti hai

↓

Pipeline artifact ke roop mein retain hoti hai

SBOM kyon useful hai?

Maan lo kal news aati hai:

OpenSSL ke particular version mein vulnerability mili

Security team SBOM search karke pata kar sakti hai:

Kaunse applications OpenSSL use karte hain?

Kaunsa version use karte hain?

Kaunse Docker images affected hain?

Kaunse environments mein deploy hain?

SBOM vulnerability identify nahi karti by itself. SBOM mainly **inventory aur traceability evidence** hai.

Scanner SBOM ke components ko vulnerability database ke against compare kar sakta hai.

2. SHA kya hota hai?

SHA = Secure Hash Algorithm

Code commit ya Docker image ka unique digital fingerprint.

Docker image SHA-256 digest

Docker image ECR mein push hone ke baad unique immutable digest milta hai:

sha256:94a00394bc5a8ef503fb59db0a7d0ae9e...

Isse prove hota hai ki QA mein tested exact image hi production mein deploy hui. Docker documentation ke according digest immutability, integrity aur different environments mein consistency provide karta hai.

Memory trick

Git SHA → Kaunsa source-code version?

Image SHA → Kaunsi exact Docker image?

3. CVE kya hota hai?

CVE = Common Vulnerabilities and Exposures

Publicly known security vulnerability ko diya hua unique identification number.

Example format: CVE-2026-12345

CVE identifier security teams, developers, vendors aur scanning tools ko ensure karta hai ki sab log exactly same vulnerability discuss kar rahe hain. NIST CVE ko publicly known vulnerabilities ke unique/common identifiers ke roop mein describe karta hai.

Docker image mein CVE kahan se aa sakti hai?

Aapke Dockerfile ki first line hai:

FROM python:3.12.11-slim

Ye image apne andar kuch operating-system packages la sakti hai:

Debian packages

OpenSSL

libc

Certificate packages

Python runtime libraries

Application dependencies:

FastAPI

Uvicorn

Pydantic

Starlette

Inmein se kisi package ke installed version mein known vulnerability hui, toh Trivy CVE report kar sakta hai:

Package: openssl

Installed version: x.y

Vulnerability: CVE-2026-12345

Severity: HIGH

Fixed version: x.z

CVE aur severity alag cheezein hain

CVE ID → Vulnerability ki identity

Severity → Vulnerability kitni serious hai

Common severity levels: CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN

Har CVE ka practical risk sirf severity se decide nahi hota. Team ye bhi dekhti hai:

Vulnerable code application mein reachable hai?

Exploit publicly available hai?

Container internet-facing hai?

Fixed version available hai?

Compensating control present hai?

Vulnerability CISA KEV catalogue mein actively exploited hai?

4. SLA kya hota hai?

SLA = Service Level Agreement

Kisi service, incident ya security issue ko defined time aur agreed service level mein handle karne ka formal commitment.

Vulnerability-remediation SLA

DevSecOps context mein hum commonly remediation timeline ki baat karte hain.

Example organization policy:

Critical vulnerability: 24 or 48 hours mein fix

High vulnerability: 7 days mein fix

Medium vulnerability: 30 days mein fix

Low vulnerability: 60 or 90 days mein fix

Ye values universal nahi hain. Har company apne:

Risk appetite

Compliance requirements

Application criticality

Internet exposure

Exploitability

Customer obligations

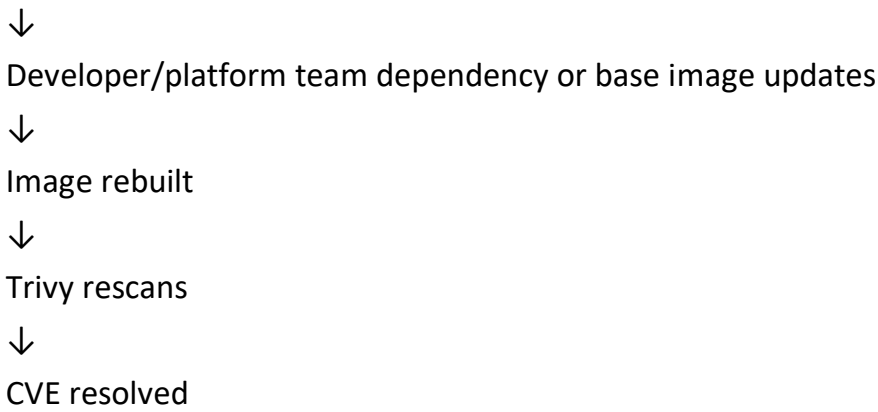
ke according SLA define karti hai.

Pipeline mein SLA ka role

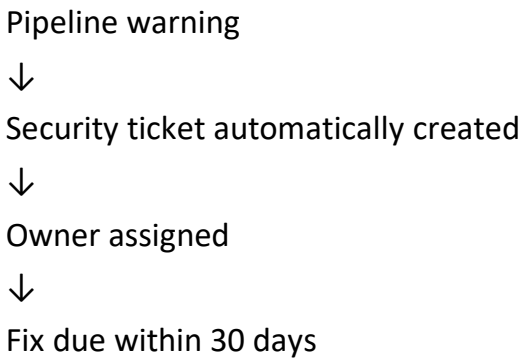
Example: Trivy detects Critical CVE

↓

Pipeline immediately blocks deployment



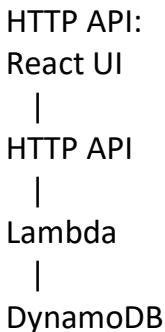
Medium issue ke liye company policy ho sakti hai:



Point	pr-validation.yml	build-and-push-dev.yml
Kab chalegi?	PR raise/update par	PR merge ke baad main push par
Kaunsa code?	Proposed feature-branch code	Exact merged main commit
Docker build?	Haan	Haan
Trivy scan?	Haan	Haan
SBOM?	Haan	Haan
AWS credentials?	Nahi	OIDC temporary credentials
ECR push?	Nahi	Haan
Image permanent?	Nahi	Haan, ECR mein
ECS deploy?	Nahi	Abhi nahi, next milestone
Role	Validation	Official DEV artifact publication

HTTP API aur REST API dono API Gateway ke types hain, lekin:

HTTP API = Simple, fast, low-cost
REST API = Advanced features, enterprise use cases
Maan lo tumhari insurance application hai:



Simple CRUD operations:

Create Claim
Get Claim
Update Claim

Bas request aayi aur Lambda ko forward kar di.

REST API
React UI
|
REST API
|
Authorizer
|
Request Validation
|
Throttling
|
Lambda
|
Database

Yahan extra controls hain:

API Keys
Usage Plans
Request Validation
Response Transformation
Advanced Security

Feature Comparison

HTTP API: Cheaper, Faster, Easy setup, JWT/OIDC Support, Lambda Integration, AWS Service Integration
REST API: API Keys, Usage Plans, Request Validation, Request/Response Mapping, Advanced Throttling, Enterprise-level controls, Mature and feature rich

RDS proxy ka kya use hai?

RDS Proxy ka main use database connections ko manage aur reuse karna hai, especially jab Lambda, ECS ya EKS se RDS/Aurora connect hota hai.

RDS Proxy application aur RDS database ke beech connection manager ki tarah kaam karta hai.

Maan lo insurance application mein ek saath 1,000 Lambda executions aa gayin. Agar har Lambda RDS ke saath apna naya connection banaye, to database par bahut saare connections open ho jayenge.

RDS ki maximum connection limit cross hone par:

- .New requests fail ho sakti hain
- .Database CPU aur memory consume hoti hai
- .Too many connections error aa sakta hai
- .Application unavailable ho sakti hai

RDS Proxy kya karta hai?

RDS Proxy pehle se database connections ka connection pool maintain karta hai. Application ke multiple requests ke liye connections reuse kiye jate hain, isliye har request ko naya database connection establish nahi karna padta.

SRE: Site Reliability Engineering.

SRE ka kaam production application ko reliable rakhna hai, lekin manual support ke bajay engineering, automation aur measurable reliability targets ke through.

Sirf application deploy kar dena **DevOps activity** ho sakti hai.

Lekin production mein yeh ensure karna ki:

- Claim API available rahe
- Response time acceptable rahe
- ECS task failure ke baad automatically replace ho
- RDS failure mein standby activate ho
- User-impacting error par alert mile
- Incident ka root cause identify ho
- Same incident dobara na ho
- Deployment se reliability damage na ho

yeh sab **SRE approach** ka part hai.

SRE ka focus

SRE production reliability ko measurable aur engineering-driven banata hai:

- Availability
- Latency
- Error rate
- Capacity
- Incident response
- SLI, SLO aur error budget
- Alerting
- On-call
- Toil reduction
- Automated recovery
- Postmortem

SRE activity:

ECS deployment

-> Health monitoring

-> SLO measurement

-> Error-budget tracking

-> User-impacting alerts

-> Automatic recovery

-> Incident response

-> Postmortem

-> Reliability improvement

SRE ke sabse important concepts

1 SLA: Service Level Agreement

SLA customer aur service provider ke beech formal ya contractual commitment hota hai.

Example: Claim Processing API will maintain 99.9% monthly availability.

Agar SLA violate hota hai toh:

- Financial penalty
- Service credits
- Contractual escalation
- Customer dissatisfaction

ho sakta hai.

2 SLO: Service Level Objective

SLO organization ka internal reliability target hota hai.

Example:

99.95% of Claim API requests should succeed during a rolling 30-day period.

SLO deliberately SLA se stricter rakha ja sakta hai, taaki customer-facing commitment violate hone se pehle engineering team ko safety margin mil jaaye.

3 SLI: Service Level Indicator

SLI actual measured performance hai.

Example:

Successful requests = 99,950

Total requests = 100,000

Availability SLI = $99,950 / 100,000 \times 100 = 99.95\%$

SLO target hai, SLI actual measurement hai.

Possible Claim API SLIs:

- Successful-request percentage
- Requests completed under 500 ms
- Claims processed correctly
- Queue messages processed within five minutes
- Claim status freshness
- Successful database writes

4 SLA, SLO aur SLI ko ek saath samjhiye

SLA: Customer se commitment = 99.9%

SLO: Internal engineering target = 99.95%

SLI: Actual measured availability = 99.97%

Is example mein service healthy hai kyunki actual SLI internal SLO se bhi better hai.

Error Budget kya hota hai?

Koi practical distributed system permanently 100% reliable nahi rehta. Agar SLO 99.9% hai, toh remaining 0.1% failure allowance error budget hai.

Error Budget = 100% - SLO

SLO = 99.9%

Error Budget = 0.1%

30-day month mein:

30 days × 24 hours × 60 minutes = 43,200 minutes

0.1% of 43,200 = 43.2 minutes

Matlab approximately **43 minutes 12 seconds** ki monthly unavailability allowed hai, assuming calendar-time availability.

Kya ALB ki jagah NLB use kar sakta hoon?

Haan, technically kar sakte ho, lekin har situation mein nahi.

Agar aapki application HTTP/HTTPS web application hai

Jaise:

- React UI
- Angular UI
- Insurance Portal
- E-commerce Website

Toh normally **ALB (Application Load Balancer)** use karte hain.

Kyun?

ALB samajhta hai:

- URL Path
- Host Name
- HTTP Headers
- Cookies

Example:

/claims --> Claim Service

/policy --> Policy Service

/payment --> Payment Service

ALB yeh routing kar sakta hai.

NLB mein problem kya hai?

NLB ko URL nahi dikhta.

NLB sirf yeh dekhta hai:

IP Address

Port Number

TCP Connection

Isliye yeh nahi bol sakta:

/claims --> Service A

/policy --> Service B

TCP kya hota hai?

Jab bhi do systems data exchange karte hain, kuch communication rules hote hain.

TCP unhi rules ka ek protocol hai.

Example

Aap WhatsApp par message bhejte ho.

TCP ensure karta hai:

- ✓ Message poora pahucha
- ✓ Correct order mein pahucha
- ✓ Koi packet missing toh dubara bhejo

TCP Example

Aap bhejte ho:

Hello

Network mein tod diya gaya:

Packet 1 = He

Packet 2 = ll

Packet 3 = o

TCP ensure karega: He + ll + o

Correct order mein receive ho.

Isliye TCP use hota hai

- Websites
- Banking
- APIs
- Database Connections
- Email

Kyuki data loss acceptable nahi hota.

UDP kya hota hai?

TCP ka fast cousin 😊

UDP nahi check karta:

- Packet mila ya nahi
- Order sahi hai ya nahi

Bas bhej deta hai.

Use cases:

- Online Gaming

- Live Video
- Video Calls
- Streaming

Kyuki speed reliability se zyada important hoti hai.

Layer 4 kya hota hai?

Networking ko OSI Model mein divide kiya gaya hai.

Interview ke liye bas yeh yaad rakho:

Layer 7 = Application Layer

Layer 4 = Transport Layer

Layer 7 (ALB)

ALB HTTP request ko samajhta hai.

Example:

HTTP

GET /claims

Host: insurance.com

ALB dekh sakta hai:

- URL
- Path
- Header
- Cookie

Isliye intelligent routing karta hai.

Layer 4 (NLB)

NLB sirf dekh raha hai:

Source IP

Destination IP

Port

TCP/UDP

Bas.

Usko yeh nahi pata:

/claims

/policy

/payment

Kya hai.

ALB aur NLB mein difference kya hai?

Aap bol sakte ho:

ALB Layer 7 load balancer hai jo HTTP/HTTPS traffic ko understand karta hai aur path-based ya host-based routing provide karta hai. NLB Layer 4 load balancer hai jo TCP/UDP traffic ko handle karta hai aur high throughput, low latency aur static IP requirements ke liye use kiya jata hai.

Event-Driven Architecture (EDA) kya hai?

Event-Driven Architecture ek aisi architecture pattern hai jisme systems **events (kuch hone ki information)** ke basis par react karte hain.

Real-Life Example

Socho aapne Amazon par order place kiya.

Event:

Order Created

Is event ke baad automatically:

- Payment Service payment process karti hai
- Inventory Service stock update karti hai
- Notification Service email bhejti hai
- Shipping Service shipment create karti hai

Event kya hota hai?

"Kuch important hua hai."

Examples:

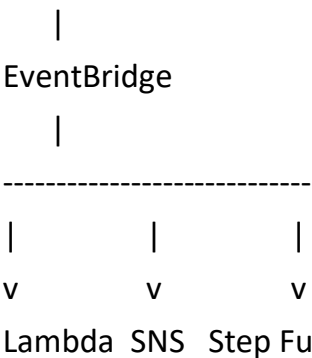
- User Registered
- Claim Submitted
- Policy Created
- Payment Completed
- File Uploaded
- Order Placed

AWS mein Event-Driven Architecture kaise banate hain?

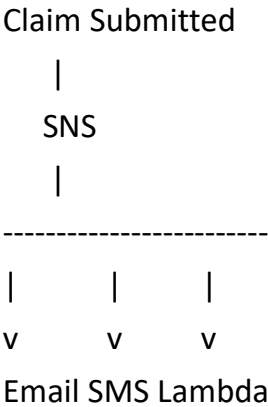
Common services:

EventBridge : Event Router

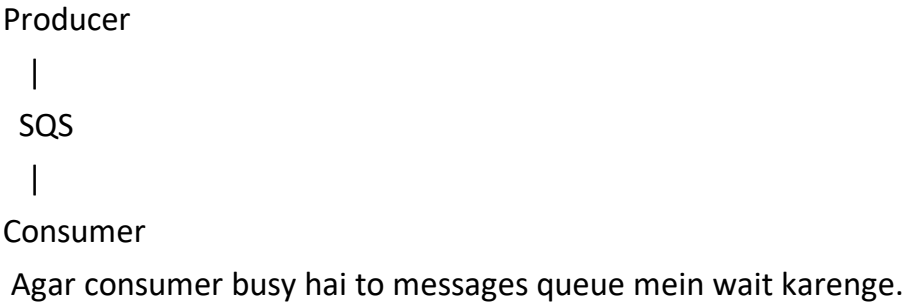
Claim Submitted



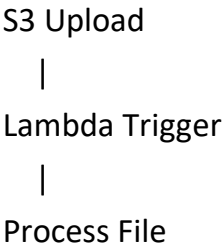
SNS: One-to-Many Notification



SQS: Queue



Lambda: Event aate hi code execute.

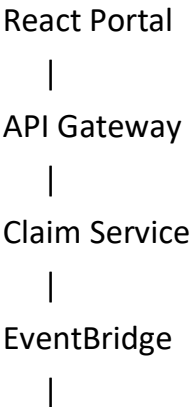


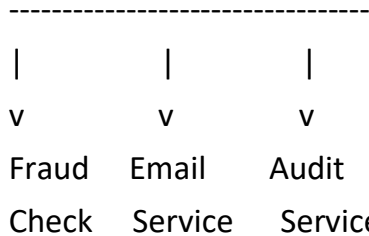
Insurance Claim Project Example

Maan lo customer claim submit karta hai.

Event
Claim Submitted

Architecture





Benefits

1. **Loose Coupling** : Claim Service ko Notification Service ka direct knowledge nahi chahiye.
2. **Scalability**: Fraud Check busy ho to Notification Service par effect nahi padega.
3. **Reliability**: Queue mein events store ho sakte hain.
4. **Easy Expansion**

Kal naya service add karna hai:

Analytics Service

Bas event subscribe kar do.

Event-Driven Architecture is a design pattern where services communicate through events rather than direct synchronous calls. A producer publishes an event, and interested consumers react to it independently. In AWS, I implement this using EventBridge, SNS, SQS, Lambda, and Step Functions to build scalable and loosely coupled systems.

AWS Lambda is a serverless compute service that allows me to run code without provisioning or managing servers. I upload my code, configure a trigger, and Lambda automatically executes the code when the event occurs. Serverless ka matlab servers exist nahi karte aisa nahi hai. AWS servers manage karta hai. I only focus on writing and deploying code while AWS handles provisioning, scaling, patching, and availability.

Lambda ko trigger kaun kaun kar sakta hai?

Common triggers:

- API Gateway
- EventBridge
- S3
- DynamoDB Streams
- SQS
- SNS
- CloudWatch Events
- Step Functions

Example: In my projects, I commonly use API Gateway to invoke Lambda for REST APIs and EventBridge for event-driven workflows.

Cold Start kya hota hai?

A cold start occurs when AWS creates a new execution environment for a Lambda function. During this initialization phase, there can be additional latency before the function starts processing requests.

Kaise reduce karte hain?

- Provisioned Concurrency
- Lightweight packages
- Efficient initialization code

Warm Start kya hota hai?

If a Lambda invocation uses an already initialized execution environment, it is called a warm start. Warm starts are faster because initialization has already been completed.

Lambda ki maximum execution time kitni hai?

AWS Lambda supports a maximum execution timeout of 15 minutes per invocation.

Lambda aur EC2 mein difference?

Lambda:

- Serverless
- Auto scaling
- Pay per execution
- No server management

EC2:

- Server management required
- Long-running applications
- Full OS control

I use Lambda for event-driven and short-duration workloads, while I use EC2 for long-running applications requiring operating system level control.

Lambda scaling kaise karti hai?

Lambda automatically scales horizontally by creating multiple execution environments to process concurrent requests. No manual scaling configuration is required in most scenarios.

Concurrency kya hai?

Concurrency represents the number of Lambda invocations running simultaneously.

Example:

1000 requests

1000 concurrent executions

Possible if account concurrency limits allow.

Reserved Concurrency kya hai?

Reserved concurrency guarantees a specific number of concurrent executions for a function and prevents other functions from consuming those resources.

Example:

Claim Lambda = 100

Policy Lambda = 50

Provisioned Concurrency kya hai?

Provisioned Concurrency keeps execution environments pre-initialized, helping reduce cold-start latency for critical applications.

Lambda environment variables kya hote hain?

Environment variables allow me to store configuration values outside the application code, such as database URLs, API endpoints, or feature flags.

Example:

DB_HOST

REGION

TABLE_NAME

Lambda ke saath VPC kyun use karte hain?

I place Lambda inside a VPC when it needs private access to resources such as RDS databases, internal services, or private endpoints.

Example:

Lambda

|

Private Subnet

|

RDS

Lambda stateless kyun hoti hai?

Lambda execution environments are temporary and should not be relied upon for persistent storage.

Therefore, application state should be stored externally in services like DynamoDB, RDS, ElastiCache, or S3.

Lambda monitoring kaise karte ho?

Services:

- CloudWatch Logs
- CloudWatch Metrics
- CloudWatch Alarms
- X-Ray

I use CloudWatch Logs for debugging, CloudWatch Metrics and Alarms for monitoring, and AWS X-Ray for distributed tracing.

Lambda error handling kaise karte ho?

I implement try-catch logic inside the function, send errors to CloudWatch Logs, and use DLQs or retry mechanisms depending on the event source.

Dead Letter Queue (DLQ) kya hai?

A Dead Letter Queue stores failed events that could not be processed successfully after all retry attempts have been exhausted.

Usually:

Lambda

|

Error

|
SQS DLQ

Lambda aur SQS saath mein kyun use karte hain?

SQS acts as a buffer between producers and consumers. Lambda polls messages from the queue and processes them asynchronously, improving reliability and scalability.

Lambda aur EventBridge saath mein kyun use karte hain?

EventBridge routes events from producers to Lambda functions. This helps build loosely coupled event-driven architectures.

Production mein Lambda deploy kaise karte ho?

I deploy Lambda through CI/CD pipelines using Git, CodePipeline, CodeBuild, and infrastructure as code tools like Terraform. I also use Lambda versions and aliases for controlled releases.

Lambda Versions aur Aliases kya hote hain?

Version: v1, v2, v3

Alias: DEV, QA, PROD

Example: PROD -> v3

Insurance Claim Project mein Lambda kaise use ki?

In my insurance claim workflow, API Gateway invokes a Lambda function that validates claim details and stores claim data in DynamoDB. After successful submission, the function publishes an event to EventBridge, which triggers downstream services such as fraud checks, notifications, and audit logging. This architecture keeps services loosely coupled and scalable.

API Gateway kya hai?

Amazon API Gateway is a fully managed service that allows me to create, publish, secure, monitor, and manage APIs. It acts as the entry point for client applications and routes requests to backend services such as Lambda, ECS, EC2, or other AWS services. API Gateway acts as a single entry point for APIs. It provides routing, authentication, authorization, throttling, monitoring, caching, and request/response transformation capabilities.

Architecture: Client -> API Gateway -> Lambda / ECS / EC2

Stages kya hote hain? Stages environment represent karte hain. Example: DEV, QA, PROD

API Gateway Authentication kaise karta hai?

Options: IAM, Cognito, JWT Token, Lambda Authorizer

Example: User Login -> JWT Token -> API Gateway

Lambda Authorizer kya hota hai?

Lambda Authorizer is a custom authorization mechanism where a Lambda function validates a token before allowing access to an API.

API Keys kya hote hain?

API Keys help identify API consumers and are commonly used along with usage plans to control API consumption.

Usage Plan kya hota hai?

Usage Plan define karta hai: Request limits, Quotas, Throttling rules

Example: 1000 requests/day

Throttling kya hoti hai?

Throttling limits the number of requests that a client can send within a given time period to protect backend systems.

Example: 100 Requests/Second

Agar user: 500 Requests/Second

API Gateway Caching kya hai?

API Gateway caching stores responses temporarily so repeated requests can be served faster without reaching the backend.

API Gateway Security Best Practices?

HTTPS only, JWT authentication, Cognito integration, IAM policies, WAF integration, Request validation, Throttling

API Gateway Private API kya hoti hai?

Private APIs are accessible only from inside a VPC using VPC endpoints and are not exposed to the public internet.

Architecture: VPC -> Private API Gateway -> Backend Services

Serverless application kaun si hoti hai

Serverless Application woh application hoti hai jisme mujhe servers manage nahi karne padte. AWS automatically infrastructure, scaling, patching aur availability handle karta hai.

Step Functions kya hai?

AWS Step Functions ek workflow orchestration service hai jo multiple services (Lambda, ECS, SNS, SQS, DynamoDB, EventBridge, etc.) ko ek defined sequence mein execute karne mein help karti hai.

Socho aapko claim process karna hai:

Step 1: Validate Claim

Step 2: Fraud Check

Step 3: Approve Claim

Step 4: Send Email

In sab ko manage karne ke liye Step Functions use kar sakte ho.

With Step Functions

Step Functions

|

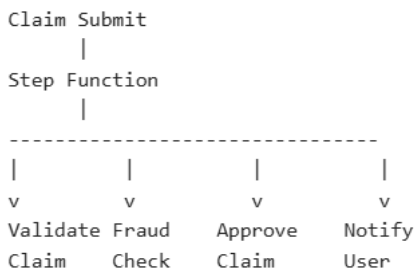
| | | |

v v v v

A -> B -> C -> D

Real Insurance Project Example

Customer claim submit karta hai.



Step Functions kyun use karte ho?

I use AWS Step Functions to orchestrate complex workflows involving multiple services. It provides visual workflow management, retry mechanisms, error handling, branching logic, and monitoring capabilities.

Step Function State Machine kya hoti hai?

A State Machine is the workflow definition in Step Functions. It contains a series of states and transitions that define how a process executes.

Example: Start -> Validate -> Approve -> Notify -> End

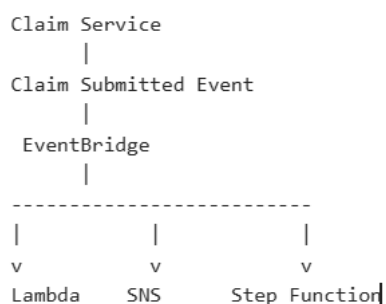
Easy Memory Trick

- **Lambda = Ek kaam**
- **Step Functions = Poora workflow**
- **EventBridge = Event routing**
- **SQS = Queue**
- **SNS = Fan-out notification**
- **Step Functions = Orchestrator / Conductor of services**

EventBridge Kya Hai?

EventBridge ek event router hai. Jab koi service event generate karti hai, EventBridge us event ko sahi destination tak pahunchata hai.

Example:



EventBridge khud business logic execute nahi karta.

Uska kaam hai:

- Event receive karna
- Event filter karna
- Event route karna

Insurance project mein customer claim submit karta hai.

Claim Submitted

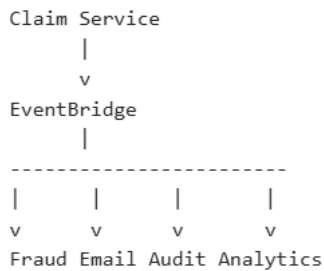
EventBridge ye event receive karega aur:

Fraud Check Lambda

Email Service

Audit Service
Analytics Service

sabko inform kar dega.



Amazon EventBridge is a serverless event bus service that enables event-driven communication between applications, AWS services, and SaaS applications. It receives events from producers and routes them to appropriate targets based on defined rules.

Event Bus kya hota hai?

Event Bus ek central location hai jahan events aakar land karte hain.

Producer -> Event Bus -> Consumers

Producer aur Consumer kya hai?

Producer

Event generate karta hai.

Examples: Claim Service, Lambda, S3, DynamoDB

Consumer

Event process karta hai.

Examples: Lambda, Step Functions, SNS, SQS

Cognito Kya Hai?

Amazon Cognito AWS ka Identity and Authentication service hai jo users ko signup, login, logout aur authorization provide karta hai.

Real Insurance Project Example

Customer insurance portal mein login karta hai.

Prashant – Login – Cognito - JWT Token - API Gateway - Lambda/ECS

Token valid hai to access milega. / Token invalid hai to access deny.

Cognito ke Major Components Kaun Se Hain?

1. User Pool

User authentication: Signup, Login, Logout, MFA, Password Reset

2. Identity Pool

AWS resources access karne ke liye temporary AWS credentials provide karta hai.

User – Cognito - Temporary AWS Credentials - S3 Access

User Pool: "Who are you?"

Identity Pool : "What can you access?"

JWT Token Kya Hota Hai?

Login ke baad Cognito token generate karta hai.

User Login – Cognito - JWT Token

Har request mein token jata hai.

Authorization: Bearer eyJ...

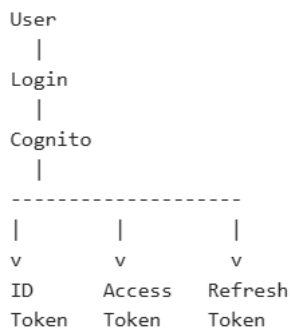
API Gateway verify karta hai.

Cognito aur API Gateway saath mein kaise kaam karte hain?

User -> Login -> Cognito -> JWT Token -> API Gateway -> Lambda

Flow:

1. User login karta hai
2. Cognito token deta hai
3. API Gateway token verify karta hai
4. Request backend tak jati hai



Cognito normally 3 tokens deta hai.

1. ID Token Kya Hai?

ID Token user ki identity information rakhta hai.

Example claims:

JSON

```
{
  "email": "user@example.com",
  "name": "Prashant",
  "sub": "123456"
}
```

Use:

- User ka naam
- Email
- Profile information

Frontend generally ise read karta hai.

2. Access Token Kya Hai?

Authorization ke liye use hota hai.

Example: HTTP, Authorization: Bearer eyJ...

API Gateway ya backend verify karta hai.

Use: Can this user access this API?

3. Refresh Token Kya Hai?

Sabse important interview topic.

Socho:

Access Token = 1 hour

1 hour baad user ko logout karna pade?

Nahi.

Refresh token naya Access Token generate kar deta hai.

Login -> Access Token Expire -> Refresh Token -> New Access Token

User ko dobara login nahi karna padta.

Access token expire ho gaya toh kya hoga?

When the access token expires, the application uses the refresh token to obtain a new access token without forcing the user to log in again.

Cognito Token Expiry Limits

Access Token

Usually: 5 Minutes - 24 Hours

Common Production: 30 Minutes, 1 Hour

ID Token

Usually: 5 Minutes - 24 Hours

Common: 1 Hour

Refresh Token

Usually: 60 Minutes - 10 Years

Common: 30 Days, 90 Days

Interview Scenario

Banking Application

Access Token = 15 mins

Refresh Token = 30 days

High security.

Internal Portal

Access Token = 1 hour

Refresh Token = 30 days

Common setup.

Cognito aur JWT validation kaun karta hai?

In production, I use Amazon Cognito User Pools for authentication and JWT token generation. Cognito issues ID, Access, and Refresh Tokens after successful login. The Access Token is used to authorize API requests, the ID Token contains user profile information, and the Refresh Token is used to obtain new Access Tokens without forcing users to log in again. I enable MFA, configure password policies, use

Cognito groups for role-based access, and integrate Cognito with API Gateway to secure backend APIs. This provides a scalable and secure authentication solution without building a custom identity platform.

Login ke baad Cognito token kahan store hota hai aur API Gateway tak kaise pahunchta hai?

Step-by-Step Flow

Step 1: User Login

React Application

|

| Username + Password

v

Cognito

Cognito response deta hai:

JSON

```
{
  "id_token": "...",
  "access_token": "...",
  "refresh_token": "..."
}
```

Step 2: Token Kahan Store Hota Hai?

Frontend application token ko store karti hai.

Common options:

Option 1: Browser Memory

JavaScript

```
const token = response.accessToken;
```

Page refresh hote hi token chala jayega.

Option 2: Session Storage

JavaScript

```
sessionStorage.setItem("token", token);
```

Benefits:

- Browser tab band hone tak available

Option 3: Local Storage

JavaScript

```
localStorage.setItem("token", token);
```

Benefits:

- Browser restart ke baad bhi available

Drawback:

- XSS attack ka risk

Option 4: Secure HTTP-Only Cookie

Production mein sabse preferred.

Browser -> HTTP Only Cookie

JavaScript cookie read nahi kar sakta.

Security better hoti hai.

Step 3: API Call Kaise Hoti Hai?

React API call karta hai:

JavaScript

```
fetch("/claims", {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```

HTTP request: HTTP, GET /claims

Authorization: Bearer eyJhbGc...

Yeh access token API Gateway tak jata hai.

Step 4: API Gateway Kya Karta Hai?

User -> JWT Token -> API Gateway

API Gateway Cognito se nahi poochta:

"Yeh user valid hai kya?"

Har request par.

Instead JWT verify karta hai.

Checks: Signature valid?

Expired toh nahi?

Issuer correct?

Audience correct?

Sab valid: 200 OK

Invalid: 401 Unauthorized

Step 5: Backend Tak Request

User -> JWT -> API Gateway -> Lambda/ECS

Real Insurance Project Example

After authentication, Cognito returns ID, Access, and Refresh Tokens. In web applications, tokens are typically stored in secure HTTP-only cookies or browser storage. During API calls, the Access Token is sent in the Authorization header as a Bearer token. API Gateway validates the JWT token issued by Cognito before forwarding the request to backend services such as Lambda or ECS. When the Access Token expires, the application uses the Refresh Token to obtain a new Access Token without requiring the user to log in again.

In production systems, I prefer storing tokens in Secure HTTP-only cookies rather than Local Storage because it reduces the risk of token theft through XSS attacks.

Real challenge + solution

One challenge I faced was related to service discovery during auto-scaling. When new microservice instances were launched, some requests were still being routed to instances that were shutting down or had become unhealthy. This caused intermittent failures and increased response times.

To address this, I configured proper health checks and readiness probes so that only healthy instances were registered in the service registry and allowed to receive traffic. I also reduced service registration and deregistration delays and implemented client-side retries with circuit breakers to handle temporary failures gracefully.

In Kubernetes, I used readiness and liveness probes along with rolling deployments to ensure traffic was routed only to healthy pods. We also enhanced monitoring and alerting around service registration events, which helped us quickly identify and resolve discovery-related issues.

As a result, we significantly reduced failed requests during deployments and scaling events, improving overall application stability and uptime.

Multi-AZ ek Region ke andar AZ failure se protect karta hai, lekin agar poora AWS Region unavailable ho jaye, to hume **cross-Region disaster recovery architecture** banana padta hai. Multi-AZ apne aap doosre Region mein database create nahi karta.

Pehle AZ aur Region ka difference

AWS Region: ap-south-1, Mumbai

- └─ AZ-1: ap-south-1a
- └─ AZ-2: ap-south-1b
- └─ AZ-3: ap-south-1c

AWS Region: ap-south-2, Hyderabad

- └─ AZ-1
- └─ AZ-2
- └─ AZ-3

Primary Region ke andar Multi-AZ

Mumbai Region

- └─ RDS Primary: AZ-1
- └─ RDS Standby: AZ-2

Yeh protection deta hai:

- Primary database instance failure
- Availability Zone failure
- Infrastructure maintenance
- Storage ya host-related failure

Standard Multi-AZ deployment synchronous replication use karta hai aur AZ-level failure par AWS standby ko automatically promote kar sakta hai. Lekin Multi-AZ **Region-wide outage se protection nahi deta**.

Region disaster ke liye kya karte hain?

Production system ke liye secondary Region mein DR environment banate hain.

Example:

Primary Region: Mumbai

Secondary DR Region: Hyderabad

Is architecture ko generally **multi-Region active-passive DR architecture** kaha jaata hai.

- Mumbai application active hai
- Hyderabad application passive, scaled down ya standby mode mein hai
- Users normally Mumbai Region ko access karte hain
- Disaster hone par Hyderabad activate hota hai

AWS guidance bhi cross-Region DR ko Region-wide outage se protection ke liye define karti hai, jabki Multi-AZ ko in-Region protection ke liye use kiya jaata hai.

Cross-Region database ke main options

Business ke **RPO, RTO aur budget** ke basis par option select hota hai.

- **RPO:** Maximum kitna recent data lose karna acceptable hai
- **RTO:** Application maximum kitni der unavailable reh sakti hai

Traffic doosre Region mein kaise jayega?

Regional ALB apne aap cross-Region failover nahi karta. Har Region ka apna ALB hota hai:

Mumbai ALB

Hyderabad ALB

Inke upar global traffic-routing mechanism chahiye.

Route 53 failover routing

Users

|

Route 53 DNS

|

+— Primary record → Mumbai ALB

|

└— Secondary record → Hyderabad ALB

Normal condition:

Route 53 → Mumbai ALB

Primary health check fail hone aur DR decision ke baad:

Route 53 → Hyderabad ALB

AWS ke multi-Region RDS blueprint mein active-passive workload ke liye primary and secondary Regions ke beech Route 53 failover policy use ki gayi hai.

Important operational detail

Route 53 health check fail hote hi blindly secondary Region activate karna database application ke liye risky ho sakta hai.

Pehle ensure karna hota hai:

1. Database secondary Region mein promoted/writable hai
2. Data replication state accepted hai
3. Application configuration new writer ko point karti hai
4. Application services healthy hain
5. Uske baad traffic switch hota hai

Warna users Hyderabad application par aa jayenge, lekin database abhi read-only ho sakta hai.

Failover ke baad wapas Mumbai kaise aayenge?

Isko **failback** kehte hain. Original Region restore hone par immediately traffic wapas nahi bhejte.

1. Mumbai Region stable verify karna
2. Hyderabad promoted DB ko current source maanna
3. Hyderabad se Mumbai ki taraf replication rebuild karna
4. Replication catch-up verify karna
5. Mumbai application test karna
6. Approved maintenance window select karna
7. Writes safely control/quiesce karna
8. Database role switch karna
9. Route 53 traffic Mumbai bhejna
10. Monitoring and reconciliation karna

Important point: Failover ke baad Hyderabad database mein new writes aa chuki hongi, isliye purane Mumbai database ko directly primary nahi bana sakte. Pehle reverse replication ya controlled data synchronization chahiye.

Interview-ready answer

For high availability within a Region, I deploy Amazon RDS using Multi-AZ, where the primary and standby are placed in separate Availability Zones with synchronous replication and automatic failover. However, Multi-AZ does not protect the application from a complete Regional outage. For Regional disaster recovery, I maintain a secondary environment in another Region using a cross-Region Read Replica, Aurora Global Database, or cross-Region backups depending on the required RPO, RTO and budget.

In a warm-standby design, the secondary Region contains the VPC, ALB, application services, configurations, secrets and a continuously replicated database. During a Regional disaster, I promote the secondary database to a writable primary, scale up the application, validate health and data consistency, and then redirect traffic using Route 53 failover routing. Because cross-Region replication is generally asynchronous, I monitor replication lag and account for possible data loss within the defined RPO. I also test both failover and failback through documented DR runbooks.

Yaad rakhne ka shortcut

Multi-AZ:

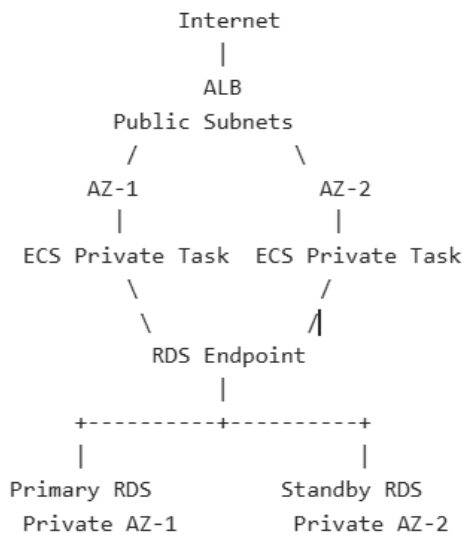
Same Region + AZ failure protection + automatic DB failover

Multi-Region:

Different Region + Regional disaster recovery + planned orchestration

Hum ALB, Private task ko multiple AZ mai deploy karte hain, kya RDS ko bhi aise hi karte hain?
Production architecture mein RDS ko bhi Multi-AZ deploy karte hain, lekin ECS private tasks ki tarah har AZ mein independently application copies nahi chalti. RDS ka Multi-AZ mechanism database replication aur automatic failover ke liye hota hai.

Overall architecture



ALB, ECS Task aur RDS mein difference

ALB : ALB ko multiple public subnets/AZs ke saath configure karte hain:

ALB node in AZ-1

ALB node in AZ-2

Purpose:

- Incoming traffic receive karna
- Healthy ECS tasks mein distribute karna
- Ek AZ down hone par doosre AZ se traffic serve karna

ECS private tasks

Same microservice ki multiple copies run karte hain:

Task 1 → Private subnet, AZ-1

Task 2 → Private subnet, AZ-2

Dono tasks simultaneously traffic process kar sakte hain.

Purpose:

- Application high availability
- Horizontal scaling
- Load distribution
- Failed task replacement

RDS Multi-AZ

Common RDS Multi-AZ instance deployment mein:

Primary RDS → AZ-1

Standby RDS → AZ-2

Application normally **Primary RDS** par read/write karti hai. RDS primary data ko synchronously standby instance par replicate karta hai. Agar primary database ya uski AZ fail ho jaye, AWS automatically standby ko new primary promote kar deta hai.

Important point: Standby active application copy nahi hai

Standard RDS Multi-AZ with one standby mein:

- Primary database read/write traffic leta hai
- Standby database failover ke liye ready rehta hai
- Application standby ko directly access nahi karti
- Standby read queries bhi serve nahi karta

AWS documentation ke mutabik, one-standby Multi-AZ deployment ka standby failover support deta hai, lekin read traffic serve nahi karta.

Isliye:

ECS Tasks: Active + Active

Standard RDS Multi-AZ: Primary + Standby

Application RDS se kaise connect karti hai?

ECS application kisi specific RDS IP ya AZ ko reference nahi karti. Application **RDS endpoint** use karti hai: insurance-db.xxxxxx.ap-south-1.rds.amazonaws.com

Application configuration:

DB_HOST = insurance-db.xxxxxx.ap-south-1.rds.amazonaws.com

DB_PORT = 5432

DB_NAME = insurance_claims

Flow:

ECS Task AZ-1 —
└— RDS Endpoint — Primary RDS

ECS Task AZ-2 —

Primary fail hone par:

Before failure: RDS Endpoint → Primary in AZ-1

After failover: RDS Endpoint → New Primary in AZ-2

Application same endpoint use karti rehti hai. Lekin active database connections temporarily break ho sakte hain, isliye application mein **connection retry, timeout aur connection pooling** properly configure karna zaroori hai.

AWS Multi-AZ with one standby deployments ke liye automatic failover, synchronous replication aur no-manual-intervention recovery provide karta hai.

RDS Subnet Group kya karta hai?

RDS create karte waqt **DB Subnet Group** banaya jaata hai. Ismein normally kam-se-kam do Availability Zones ke private database subnets select karte hain:

DB Subnet Group

└— Database private subnet in AZ-1

└— Database private subnet in AZ-2

Example VPC:

AZ-1

- └— Public subnet: ALB
- └— Private app subnet: ECS Tasks
- └— Private DB subnet: RDS Primary

AZ-2

- └— Public subnet: ALB
- └— Private app subnet: ECS Tasks
- └— Private DB subnet: RDS Standby

Security best practice:

- RDS ko public subnet mein nahi rakhna
- Publicly accessible = No
- RDS Security Group inbound mein sirf ECS Task Security Group ko allow karna

RDS Security Group inbound rule:

Type: PostgreSQL

Port: 5432

Source: ECS Task Security Group

IP address allow karne ki jagah security-group-to-security-group rule better hota hai.

Multi-AZ aur Read Replica same nahi hain

Yeh interview mein bahut important distinction hai.

Multi-AZ

Purpose: High availability + disaster/failure recovery

Standard Multi-AZ standby:

- Different AZ mein hota hai
- Synchronous replication hoti hai
- Automatic failover hota hai
- Read traffic serve nahi karta

Read Replica

Purpose: Read scaling

Example:

Primary RDS

- └— Write queries
- └— Important consistent reads
- └— Replication → Read Replica
- └— Reports
- └— Dashboard queries
- └— Analytics reads

Read Replica par read traffic bhej sakte hain, lekin replication generally asynchronous hoti hai, isliye small replication lag possible hai.

Shortcut

Multi-AZ = Availability

Read Replica = Read scalability

AWS bhi clarify karta hai ki standard Multi-AZ standby read scaling solution nahi hai. Read-only traffic ke liye Read Replica ya Multi-AZ DB cluster use kiya ja sakta hai.

Kya RDS ke teen AZ options bhi hote hain?

Haan. AWS RDS mein broadly:

Multi-AZ DB instance deployment

AZ-1: Primary

AZ-2: One standby

Standby read traffic serve nahi karta.

Multi-AZ DB cluster deployment

AZ-1: Writer

AZ-2: Readable standby

AZ-3: Readable standby

Ismein two standby DB instances:

- Failover targets hote hain
- Read traffic serve kar sakte hain
- Separate reader aur writer endpoints provide kiye ja sakte hain

AWS documentation ke according Multi-AZ DB cluster deployment mein three DB instances across three AZs hote hain, jisme writer aur two readable standby instances hote hain.

Aapke insurance microservice ke liye recommended production layout

VPC

|

└─ AZ-1

| └─ Public subnet: ALB

| └─ Private app subnet: ECS Task

| └─ Private DB subnet: RDS Primary

|

└─ AZ-2

└─ Public subnet: ALB

└─ Private app subnet: ECS Task

└─ Private DB subnet: RDS Standby

Request flow:

User -> ALB across two AZs -> Healthy ECS private task -> RDS endpoint -> Current Primary RDS -> Synchronous replication to standby

Interview-ready answer

Yes, I deploy Amazon RDS using a Multi-AZ configuration for production workloads. I configure a DB subnet group containing private database subnets from multiple Availability Zones. RDS provisions a primary database in one AZ and a synchronously replicated standby in another AZ. The application connects using the RDS endpoint rather than a database IP address. If the primary database or Availability Zone fails, RDS automatically promotes the standby and redirects the endpoint. I use Multi-AZ for high availability, while I use read replicas separately when the requirement is read scalability.

90-Second Interview Version

I design highly available and fault-tolerant AWS applications by first defining the availability target, critical dependencies, RTO, and RPO. I then eliminate single points of failure by deploying the workload across at least two Availability Zones.

I use Route 53, CloudFront, and an Application Load Balancer to route traffic to healthy application instances or containers. At the compute layer, I use EC2 Auto Scaling, ECS Fargate, or EKS with health checks, horizontal scaling, and automatic replacement of unhealthy resources.

I keep application services stateless and store state in resilient managed services. For relational databases, I use RDS Multi-AZ or Aurora with automatic failover. I also configure automated backups, point-in-time recovery, and cross-Region backup copies where required.

To prevent cascading failures, I decouple services using SQS, SNS, or EventBridge and implement timeouts, retries with exponential backoff, circuit breakers, idempotency, and dead-letter queues.

I monitor application and infrastructure health using CloudWatch metrics, logs, dashboards, alarms, tracing, ALB health checks, and synthetic monitoring. For deployments, I use rolling, Blue/Green, or canary releases with automated validation and rollback.

Multi-AZ is my default design for Regional high availability. If the business cannot tolerate a complete Regional outage, I select an appropriate multi-Region disaster-recovery strategy based on RTO, RPO, cost, and complexity. I also regularly test failover, backup restoration, and recovery runbooks because an untested recovery strategy cannot be considered reliable.

Aap apne insurance project se is answer ko connect karke aur credible bana sakte hain:

In my insurance platform, I deployed the Claim and Policy microservices as separate ECS Fargate services across multiple Availability Zones. An Application Load Balancer distributed traffic between healthy tasks and used path-based routing to send claim and policy requests to their respective target groups.

Each ECS service had independent task definitions, health checks, scaling policies, CloudWatch logs, deployment controls, and rollback capability. If one task failed, ECS replaced it automatically. If an Availability Zone became unavailable, the load balancer could route traffic to healthy tasks in another Availability Zone.

I also configured the services so that one microservice failure did not unnecessarily bring down the complete platform. For asynchronous processing, I could use SQS with retries and a dead-letter queue. For synchronous Claim-to-Policy communication, I used controlled timeouts, error handling, and graceful responses when the Policy Service was temporarily unavailable.

For production releases, I built an immutable image, stored it in ECR, registered a new ECS task-definition revision, and performed a controlled ECS deployment. The ALB sent traffic to the new tasks only after they passed health checks. ECS drained the old tasks after the new tasks became healthy, and deployment rollback controls protected the service if the new revision failed.

Configuring a load balancer across at least two Availability Zones, with registered targets in each enabled zone, improves fault tolerance during target or zonal failures.

Best Interview Answer

To design a highly available and fault-tolerant architecture for a critical web application on AWS, I first identify the application's availability target, critical user journeys, dependencies, and recovery requirements such as RTO and RPO. Based on these requirements, I design the workload to tolerate failures at the instance, container, Availability Zone, and, where required, Region level.

My first principle is to eliminate single points of failure. I deploy production workloads across at least two Availability Zones within an AWS Region. For internet-facing applications, I use Route 53 for DNS, CloudFront for global content delivery, AWS WAF for application-layer protection, and an Application Load Balancer to distribute requests across healthy application instances or containers in multiple Availability Zones.

At the compute layer, I run stateless application components using EC2 Auto Scaling, ECS Fargate, or EKS across multiple Availability Zones. I configure health checks, minimum healthy capacity, scaling policies, and self-healing so that unhealthy instances, containers, or pods are automatically replaced. I also make sure that the remaining Availability Zones have sufficient capacity to handle traffic if one Availability Zone becomes unavailable.

I keep application services stateless wherever possible. Sessions, files, and application state are stored outside the compute layer using services such as ElastiCache, DynamoDB, Amazon S3, or a resilient database. This allows any healthy application instance or container to process the request and makes horizontal scaling and replacement much easier.

For the database layer, I select the service based on the workload. For relational workloads, I use Amazon RDS Multi-AZ or Amazon Aurora across multiple Availability Zones. This provides a synchronous standby or replicated database instances and supports automatic failover if the primary database instance or its Availability Zone becomes unavailable.

Read replicas can be used to scale read traffic, but I do not treat an ordinary read replica as a direct replacement for Multi-AZ high availability. Multi-AZ primarily provides availability and failover, while read replicas primarily provide read scalability, although some Aurora architectures can support both requirements.

I enable automated backups, point-in-time recovery, appropriate backup retention, and AWS Backup policies. For critical workloads, I also protect backups against accidental deletion by using separate backup controls, encryption, access restrictions, and cross-Region or cross-account copies where required.

To prevent failures from cascading between microservices, I decouple components using Amazon SQS, SNS, or EventBridge. SQS acts as a durable buffer when a downstream service is temporarily slow or unavailable. I configure retries with exponential backoff and jitter, request timeouts, circuit-breaker

patterns, idempotent processing, and dead-letter queues so that failed messages can be isolated and reprocessed safely.

I also design the application for graceful degradation. For example, if a non-critical recommendation or reporting service becomes unavailable, the core claim or policy operation should continue wherever possible instead of failing the complete customer transaction.

For observability, I configure CloudWatch metrics, logs, dashboards, alarms, and distributed tracing. I monitor infrastructure and application-level indicators such as target health, error rate, latency, request volume, throttling, queue depth, database connections, CPU, memory, and failed business transactions. CloudWatch alarms can send notifications through SNS and trigger automated recovery actions where appropriate. I also use ALB health checks, Route 53 health checks, synthetic monitoring, ECS deployment circuit breakers, and service-stability checks to detect failures and prevent traffic from reaching unhealthy components.

For application releases, I use automated CI/CD pipelines with rolling, Blue/Green, or canary deployment strategies based on the workload's risk. New versions are validated through health checks, smoke tests, integration tests, and monitoring before receiving full production traffic. If error rate or latency crosses the defined threshold, the deployment is automatically stopped or rolled back to the last stable version.

Multi-AZ is my default strategy for high availability within a Region. For workloads where a complete Regional outage cannot be accepted, I add a cross-Region disaster-recovery strategy such as backup and restore, pilot light, warm standby, or active-active. The selected strategy depends on the business RTO, RPO, complexity, and cost requirements.

For cross-Region recovery, I can use Route 53 failover routing or Global Accelerator, database replication such as Aurora Global Database or DynamoDB global tables, S3 Cross-Region Replication, replicated container images, and Infrastructure as Code to provision consistent infrastructure in the recovery Region. Finally, I do not assume that redundancy automatically provides resilience. I regularly test instance failure, task failure, Availability Zone failure, database failover, dependency failure, deployment rollback, backup restoration, and disaster-recovery procedures. I document the recovery runbooks and validate whether the architecture actually meets its agreed RTO, RPO, and availability targets.

Therefore, my overall strategy combines Multi-AZ deployment, load balancing, self-healing compute, resilient data storage, service decoupling, fault isolation, monitoring, automated recovery, safe deployments, backups, and tested disaster recovery. This allows the application to continue serving critical functionality even when individual components or an Availability Zone fail.